Módulo	Lenguajes de Marca y Sistemas de Gestión de Información
Ciclo	Desarrollo de Aplicaciones Web (DAW) — Ciclo Superior
Modalidad	Por parejas
Duración estimada	2 semanas

1. Contexto del proyecto

La Alianza Rebelde necesita urgentemente un sistema de gestión web para coordinar sus operaciones contra el Imperio Galáctico. Como desarrollador/a web de la Alianza, tu misión es construir una aplicación completa usando HTML, CSS y JavaScript puro (sin frameworks) que permita a los rebeldes gestionar sus naves, pilotos y misiones desde un único panel de control.

La web debe ser funcional, visualmente coherente con el universo Star Wars y demostrar un dominio sólido de la manipulación del DOM.

2. Objetivos de aprendizaje

- Manipular el DOM de forma dinámica mediante JavaScript puro.
- Gestionar datos en memoria usando estructuras de objetos y arrays.
- Implementar un CRUD completo (crear, leer, actualizar, eliminar) sin base de datos.
- Aplicar persistencia con localStorage para que los datos no se pierdan al recargar.
- Diseñar una interfaz coherente y navegable con HTML y CSS.
- Estructurar el código JavaScript en módulos o secciones bien organizadas.
- La aplicación debe cumplir con los niveles de accesibilidad **AA** según la normativa **WCAG**.

3. Requisitos del proyecto

3.1 Estructura general de la web

La web contará con una barra de navegación superior fija que permita moverse entre las cuatro secciones principales. Todas las secciones deben existir en el mismo documento HTML (single-page), mostrando y ocultando contenido mediante JavaScript.

3.2 Sección 1 — Hangar de naves

Esta sección mostrará un catálogo de naves de la Alianza Rebelde. Los datos de las naves se almacenarán en un array de objetos JavaScript con las siguientes propiedades mínimas:

- nombre (ej. X-Wing, Millennium Falcon)
- tipo (caza, transporte, fragata…)
- velocidad en megaluz
- tripulación máxima
- estado (operativa / en reparación / destruida)
- imagen o emoji representativo

Funcionalidades requeridas en el hangar:

- Las tarjetas de nave se generan dinámicamente desde el array (sin HTML estático).
- Buscador en tiempo real que filtra por nombre mientras el usuario escribe (evento input).
- Menú desplegable para filtrar por tipo de nave.
- Botón para ordenar las naves por velocidad (ascendente / descendente, alternando al hacer clic).
- Contador visible que muestra cuántas naves se están mostrando en cada momento.

3.3 Sección 2 — Registro de pilotos

Gestión completa de los/as pilotos de la Alianza. El alumnado implementará un CRUD completo:

- Formulario para añadir un/a piloto nuevo/a con los campos: nombre, rango, nave asignada (select dinámico generado desde el array de naves), victorias en combate y estado (activo / herido / KIA).
- Tabla o lista de pilotos generada dinámicamente desde un array de objetos.
- Botón Editar en cada piloto: rellena el formulario con sus datos actuales para modificarlos.
- Botón Eliminar en cada piloto: solicita confirmación antes de borrar.
- Validación del formulario: ningún campo puede quedar vacío; las victorias deben ser un número positivo.

- Todos los datos persisten en localStorage (al recargar la página, los/as pilotos siguen ahí).

3.4 Sección 3 — Panel de misiones (Kanban)

Un tablero Kanban con tres columnas: Pendiente, En curso y Completada. Cada misión será una tarjeta con los siguientes datos:

- nombre de la misión
- descripción breve
- piloto asignado (seleccionable de la lista de pilotos activos)
- nivel de dificultad: fácil / media / difícil / suicida
- fecha de inicio

Funcionalidades requeridas:

- Formulario para crear nuevas misiones, que se añaden siempre en la columna Pendiente.
- Botones para mover la tarjeta hacia la columna siguiente (Pendiente → En curso → Completada).
- Botón para eliminar una misión.
- Filtro por nivel de dificultad que actúa sobre las tres columnas simultáneamente.
- El encabezado de cada columna muestra el número de tarjetas que contiene en ese momento.
- Las misiones persisten en localStorage.

3.5 Sección 4 — Mando de la Alianza (Dashboard)

Panel de resumen con estadísticas generadas automáticamente a partir de los datos de las otras secciones. Debe actualizarse cada vez que el usuario navega a esta sección o modifica datos.

- Total de naves en el hangar y desglose por estado (operativas / en reparación / destruidas).
- Total de pilotos y desglose por estado (activos / heridos / KIA).
- Total de misiones por columna (pendientes / en curso / completadas).
- El/La piloto con más victorias en combate (nombre y número).
- La nave más rápida del hangar (nombre y velocidad).
- Al menos una representación visual: una barra de progreso o un gráfico sencillo generado con DOM (sin librerías externas) que muestre, por ejemplo, el porcentaje de misiones completadas.

4. Requisitos técnicos obligatorios

HTML	Un único archivo index.html bien estructurado con etiquetas semánticas (header, nav, main, section, footer). Sin frameworks ni plantillas.
CSS	Archivo styles.css externo. Diseño responsive (al menos adaptado a pantalla de escritorio). Variables CSS (--color-primario, etc.). Coherencia visual con la temática Star Wars: paleta oscura, tipografía sci-fi o sans-serif limpia.
JS	Un único archivo main.js (o varios bien organizados). JavaScript puro, sin jQuery ni librerías. Uso de const y let, nunca var. Funciones bien nombradas y comentadas. Separación clara entre datos (arrays/objetos), lógica y presentación (DOM).
DOM	El contenido de las cuatro secciones se genera exclusivamente mediante JavaScript. El HTML base solo contiene los contenedores vacíos, la navegación y el formulario.
localStorage	Los arrays de pilotos y misiones se guardan y recuperan de localStorage. Las naves pueden ser datos iniciales hardcodeados, aunque si el alumno permite añadir naves, también deberán persistir.
Calidad Software	Revisión del nivel de accesibilidad web e informe técnico dentro del documento técnico del proyecto solicitado
Despliegue	El código deberá estar desplegado en **Vercel** y deberá estar almacenado en un repositorio de GitHub

5. Entregables

- Carpeta del proyecto comprimida en .zip con la siguiente estructura:
  - index.html en la raíz
  - css/styles.css
  - js/main.js (y otros archivos .js si se han separado por módulos)
  - img/ con imágenes o recursos utilizados (si los hay)
  - README.pdf con nombre del alumno/a, descripción breve y cualquier nota relevante
- La entrega se realizará a través de la plataforma del centro antes de la fecha indicada por la profesora.
- El proyecto debe funcionar correctamente al abrir index.html directamente en el navegador, sin servidor local.
- URL del sitio web publicado
- URL del repositorio de github. IMPORTANTE QUE CADA UNO SUBA SUS CAMBIOS.
- Documento técnico formal, donde se explique la información relevante relacionada con la gestión del proyecto (Trello). Se describirá la página y se explicarán aquellas partes

técnicas más relevantes y que sirvan como reto técnico. En este proyecto también se debe explicar el proceso que se ha seguido para subir la página al hosting gratuito. El documento técnico deberá contener un apartado de “Calidad del software” en este caso se pide que se analice la accesibilidad de la web, no sirve únicamente una captura de pantalla del análisis automático realizado por alguna herramienta.

6. Rúbrica de evaluación

No podrá dejarse de hacer ninguna de las partes del proyecto. Debe haberse intentado todas las partes

Criterio	Descripción	Puntos
Hangar de naves	Catálogo generado dinámicamente, buscador en tiempo real, filtro por tipo, ordenación y contador correctos. Se deberá poder defender justificadamente el desarrollo realizado. La profesora realizará las preguntas necesarias para ello.	**2 pts**
Registro de pilotos	CRUD completo (crear, editar, eliminar), validación de formulario, select dinámico de naves y persistencia en localStorage. Se deberá poder defender justificadamente el desarrollo realizado. La profesora realizará las preguntas necesarias para ello.	**2,5 pts**
Panel de misiones	Kanban funcional con tres columnas, movimiento de tarjetas, filtro por dificultad, contadores y persistencia. Se deberá poder defender justificadamente el desarrollo realizado. La profesora realizará las preguntas necesarias para ello.	**2 pts**
Dashboard	Estadísticas calculadas correctamente y actualizadas, representación visual generada con DOM. Se deberá poder defender justificadamente el desarrollo realizado. La profesora realizará las preguntas necesarias para ello.	**1,5 pts**
Calidad del código	Código legible, bien estructurado, comentado y sin repeticiones innecesarias. Uso correcto de const/let. Se deberá poder defender justificadamente el desarrollo realizado. La profesora realizará las preguntas necesarias para ello.	**1 pt**
Diseño y CSS	Interfaz coherente, responsive, con variables CSS y estética acorde a la temática. Se deberá poder defender justificadamente el desarrollo realizado. La profesora realizará las preguntas necesarias para ello.	**1 pt**
TOTAL		**10 pts**

7. Ampliaciones opcionales (hasta +1,5 puntos extra)

Las siguientes funcionalidades son opcionales y permiten mejorar la nota final. No se penaliza por no realizarlas.

- **Drag & drop en el Kanban: arrastrar tarjetas entre columnas con la API de drag and drop del navegador. (+0,5 pts)**
- **Modo oscuro / claro: toggle que cambie el tema visual completo de la web. (+0,25 pts)**
- **Exportar a JSON: botón que descargue los datos de pilotos y misiones como archivo .json. (+0,25 pts)**
- **Animaciones CSS: transiciones al añadir/eliminar tarjetas y al navegar entre secciones. (+0,25 pts)**
- **Página de detalle de nave: al hacer clic en una tarjeta del hangar, mostrar un modal con información extendida. (+0,25 pts)**

8. Criterios generales y normas de entrega

Originalidad	El código debe ser original. Está permitido consultar documentación (MDN, etc.) y usar código de ejercicios de clase como referencia, pero no copiar proyectos de compañeros/as ni de Internet.

Frameworks	No se permite el uso de React, Vue, Angular, jQuery ni ninguna librería de componentes. **Solo JavaScript puro, HTML y CSS.**

IA generativa	Solo se permite usar herramientas de IA como apoyo para resolver dudas puntuales, pero el alumno/a debe ser capaz de explicar y defender cualquier parte de su código. El uso masivo de IA sin comprensión se considera equivalente a copia.

Entrega tardía	Las entregas fuera de plazo implicarán no superar el proyecto y obtener un 0.

Que la Fuerza del DOM te acompañe.

La entrega deberá realizarse el día 20 de abril a través de Moodle Centros.

Su defensa en clase se realizará el 21 de abril.